

C6

A

A Keyboard Magic Power

时间限制：1000ms 内存限制：65536kb

通过率：180/194 (92.78%) 正确率：180/457 (39.39%)

题目描述

the_ignorant 在写编程题时，总喜欢用键盘上的一些神秘代码来辅助编程，比如 Ctrl-C，Ctrl-V，Ctrl-F。其中，Ctrl-F 的功能是在现有的字符串 s 中查找输入字符串 t 出现的所有位置（非常适合查找原题）。

他在使用 Ctrl-F 时，却发现键盘失灵了。但万幸的是，他知道键盘中蕴含着一种神秘力量（Keyboard Magic Power）能替代 Ctrl-F 解决问题。他希望你帮帮他，去发掘出这种神秘力量吧！

输入格式

第一行一个非空字符串 s ，表示匹配串（主串）。

第二行一个非空字符串 t ，表示模式串（子串）。

两个字符串的长度均不超过 10^6 ，且由可见字符组成。

输出格式

一行从小到大若干个非负数，表示每次匹配成功后首字符的下标位置，其中下标从 0 开始。

输入样例 1

```
aaaaa
aa
```

输出样例 1

```
0 1 2 3
```

```
#include<bits/stdc++.h>
using namespace std;
const int N = 2e6;
char s[N],t[N];
int a[N];
int nxt[N];
void kmp(int n,char *a,int m,char *b) {
    int i,j;
    for(nxt[0]=j=-1,i=1; i<n; nxt[i++]=j) {
        while(~j&&a[j+1]!=a[i])j=nxt[j];
        if(a[j+1]==a[i])j++;
    }
    for(j=-1,i=0; i<m; i++) {
        while(~j&&a[j+1]!=b[i])j=nxt[j];
        if(a[j+1]==b[i])j++;
        if(j==n-1)printf("%d ",i-n+1),j=nxt[j];
    }
}
int main() {
    scanf("%s",s);
    scanf("%s",t);
    int len1=strlen(t),len2=strlen(s);
    assert(len1>=1&&len1<=1e6);
    assert(len2>=1&&len2<=1e6);
    kmp(len1,t,len2,s);
}
```

Hint: KMP

B

B 小水獭和三点共线

时间限制: 1000ms 内存限制: 65536kb

通过率: 170/188 (90.43%) 正确率: 170/520 (32.69%)

题目描述



小水獭正在补习计算几何。

二维平面上有 n 个不同的点，小水獭请你帮它判断是否存在三点不共线？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($3 \leq n \leq 10^5$)，含义同题目描述。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$)，表示第 i 个点的坐标。

保证任意两点坐标不同。

输出格式

对于每组数据，输出一行一个字符串：

- 如果存在三点不共线，输出 `how?`；
- 否则，输出 `boo how! boo how!`。

...

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n;
int x[maxn], y[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d%d", &x[i], &y[i]);
        bool tag = false;
        for(int i = 3; i <= n; i++){
            if(1ll * (x[i] - x[1]) * (y[i] - y[2]) != 1ll * (x[i] - x[2]) *
(y[i] - y[1])){
                tag = true;
                break;
            }
        }
        puts(tag ? "how?" : "boo how! boo how!");
    }
}
```

hint: 计算几何

时间限制：1000ms 内存限制：65536Kb

通过率：84/120 (70.00%) 正确率：84/269 (31.23%)

题目描述

the_ignorant 最近玩了一款名叫 **hsah** 的格斗游戏，它有一个奇怪的机制：输入一套连招后，系统会依次判断这套连招的前 i 个操作和后 i 个操作是否相同，每相同一次，伤害就会降低一次。
the_ignorant 希望你能告诉他，输入一套连招后，哪些 i 会使得伤害降低。

形式化地说，给定字符串 s ，请你求出对于 $i = 1, 2, \dots, |s|$ 而言，长度为 i 的前缀和长度为 i 后缀是否相同。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个由小写字母、大写字母和数字组成的字符串 s ($1 \leq |s| \leq 10^6$)，含义同题目描述。

输出格式

对于每组数据，输出一行从小到大若干个正整数，表示符合条件的 i 。

输入样例

```
2
aaaaa
hsah
```

输出样例

```
1 2 3 4 5
1 4
```

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 2e6;
const int P=31,D=1000173169;
int n,i,p[N],f[N];
char a[N];
int Hash(int l,int r) {
    return (ll)(f[r]-(ll)f[l-1]*p[r-l+1]%D+D)%D;
}
int main() {
    int t;
    scanf("%d",&t);
    assert(t>=1&&t<=10);
    while(t--) {
        scanf("%s",a+1);
        int n=strlen(a+1);
        assert(n>=1&&n<=1e6);
        for(p[0]=i=1; i<=n; i++) {
            p[i]=(ll)p[i-1]*P%D;
        }
        for(i=1; i<=n; i++) {
            f[i]=(ll)((ll)f[i-1]*P+a[i])%D;
        }
        for(i=1; i<=n; i++) {
            if(Hash(1,i)==Hash(n-i+1,n)) {
                printf("%d ",i);
            }
        }
        puts("");
    }
}
```

hint: hash公式

D

D BellalaBella 和回文串

时间限制: 1000ms 内存限制: 65536kb

通过率: 177/185 (95.68%) 正确率: 177/362 (48.90%)

题目描述

Bellalabella 今天学习了回文串的内容，一个字符串是回文串当且仅当将其翻转后和原串相同。

Bellalabella 想知道对于一个字符串 S ，它的最长回文子串长度是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个由小写字母组成的字符串 S ，含义同题目描述。

保证对于所有数据 $|S|$ 的和不超过 2×10^3 。

输出格式

对于每组数据，输出一行一个正整数，表示串 S 的最长回文子串长度。

输入样例

```
2
aba
aabb
```

输出样例

```
3
2
```

```
#include<bits/stdc++.h>
#define inf 0x7fffffff
#define RG register int
using namespace std;
typedef long long ll;
const ll mod = 998244353;
const long double eps = 1e-11;
//using namespace std;
inline ll read(){
    ll x=0,f=1;char ch=getchar();
    while(ch>'9' || ch<'0'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0' && ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
int n,w,h;
char ch[110005],pat[220005];
int pos,r[220005],ans;
int main(){
    int i,j,u,v;
    int t = read();
    while(t--){
        scanf("%s",ch+1);
        n=strlen(ch+1);
        pat[0]='@';
        ans = 0;
        for(int i = 0; i <= 2 * n + 2; i++) r[i] = 0;
        for(i=1;i<=n;i++) pat[2*i-1]=ch[i],pat[2*i]='@';
        pos=0,r[0]=0;
        for(i=1;i<=2*n;i++){
            if(i<=pos+r[pos]) r[i]=min(pos+r[pos]-i,r[pos*2-i]);
            while(i>r[i]&&pat[i+r[i]+1]==pat[i-r[i]-1]) r[i]++;
        }
    }
}
```

```

        if(i+r[i]>pos+r[pos]) pos=i;
        ans=max(ans,r[i]);
    }
    printf("%d\n", ans);
}
return 0;
}

```

最长回文子串

E

E 小水獭和后缀数组

时间限制: 1000ms 内存限制: 65536kb

通过率: 27/49 (55.10%) 正确率: 27/104 (25.96%)

题目描述



小水獭正在预习字符串。

给定字符串 s ，设它的长度为 $|s|$ ， s_i 表示 s 的第 i 个字符。

s 有 $|s|$ 个后缀，第 i ($1 \leq i \leq |s|$) 个后缀为 $s_i s_{i+1} \dots s_{|s|}$ ，小水獭想知道这 $|s|$ 个后缀中，字典序最小的后缀是哪一个呢？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个由小写字母组成的字符串 s ($1 \leq |s| \leq 10^5$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个字符串，表示字典序最小的后缀。

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int a[maxn], b[maxn], c[maxn];
int SA[maxn], rnk[maxn];
void build(char *s, int n, int m) {
    int *x = a, *y = b;
    s[n + 1] = 0;
    for (int i = 0; i <= n * 2 && i < maxn; i++) a[i] = b[i] = SA[i] = rnk[i] = 0;
    for (int i = 1; i <= m; i++) c[i] = 0;
    for (int i = 1; i <= n; i++) c[x[i] = s[i]]++;
    for (int i = 1; i <= m; i++) c[i] += c[i - 1];
    for (int i = n; i; i--) SA[c[x[i]] - 1] = i;
    for (int j = 1, p = 0; j <= n && p != n + 1; j <= 1) {
        p = 0;
        for (int i = n - j + 1; i <= n; i++) y[++p] = i;
        for (int i = 1; i <= n; i++) if (SA[i] > j) y[++p] = SA[i] - j;
        for (int i = 1; i <= m; i++) c[i] = 0;
        for (int i = 1; i <= n; i++) c[x[y[i]]]++;
        for (int i = 1; i <= m; i++) c[i] += c[i - 1];
    }
}

```

```
        for (int i = n; i; i--) SA[c[x[y[i]]]--] = y[i];
        std::swap(x, y); x[SA[1]] = 1; p = 2;
        for (int i = 2; i <= n; i++) x[SA[i]] = (y[SA[i]] == y[SA[i - 1]] &&
y[SA[i] + j] == y[SA[i - 1] + j]) ? p - 1 : p++;
        m = p;
    }
    for (int i = 1; i <= n; i++) rnk[SA[i]] = i;
}

int t, n;
char s[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%s", s + 1);
        n = strlen(s + 1);
        build(s, n, 256);
        printf("%s\n", s + SA[1]);
    }
}
```

F

F BellalaBella 和多项式

时间限制: 1000ms 内存限制: 65536kb
通过率: 13/14 (92.86%) 正确率: 13/23 (56.52%)

题目描述

Bellalabella 在数学课上学习了函数的知识。由于 Bellalabella 只上了初中，她只接触到了一元多项式的知识。

老师想要 Bellalabella 对于给定的多项式 $f(x) = \sum_{i=0}^{n-1} a_i x^i$ 求出其除以另一个多项式 $g(x) = x + c$ 的商 $h(x) = \sum_{i=0}^{n-2} b_i x^i$ 和余数 r ，但是 Bellalabella 实在是太笨了，她只能寻求来自小水獭幼儿园的你解答这个问题。

形式化地说，设 $f(x) = h(x) \times (x + c) + r$ ，给定 $f(x)$ 的各项系数和常数 c ，请你求出 $h(x)$ 的各项系数和常数 r 。

输入格式

第一行两个整数 n, c ($1 \leq n \leq 10^6, 0 \leq |c| \leq 10^9$)，含义同题目描述。

第二行 n 个整数 $a_0, a_1, \dots, a_{n-1}$ ($0 \leq |a_i| \leq 10^9$)，含义同题目描述。

输出格式

一行 n 个整数 $b_0, b_1, \dots, b_{n-2}, r$ ，数据保证这 n 个整数的绝对值不超过 10^9 。

输入样例

3 1
1 2 1

输出样例

1 1 0

样例解释

$f(x) = 1 + 2x + x^2 = (x + 1) \times (x + 1) = h(x) \times (x + 1) + 0$ ，因此 $b_0 = b_1 = 1, r = 0$ 。

```
#include <stdio>
#include <algorithm>
#include <cstring>

using namespace std;
```

```

const int N = 1e6 + 5;

int n;
long long c, a[N], b[N], r;

void solve()
{
    scanf("%d%lld", &n, &c);
    for (int i = 1; i <= n; i++)
        scanf("%lld", &a[i]);
    for (int i = n; i > 1; i--)
    {
        b[i - 1] = a[i];
        a[i] -= b[i - 1];
        a[i - 1] -= c * b[i - 1];
    }
    r = a[1];
    for (int i = 1; i < n; i++)
        printf("%lld ", b[i]);
    printf("%lld\n", r);
}

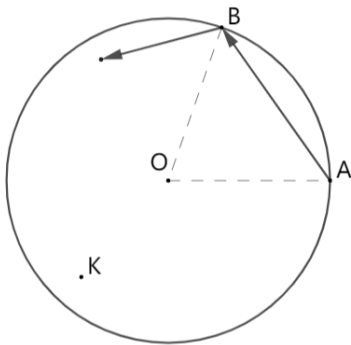
int main()
{
    int T = 1;
    // scanf("%d", &T);
    while (T--)
        solve();
    return 0;
}

```

G

题目描述

Zhoues 有一块神奇的圆形镜子，里面有一束永不消失的光。



光从圆 O 上一点 A 出发，在镜子内部不断沿逆时针反射。Zhoues 想知道在给定 $\angle ABO$ 的情况下，这一束光经过多少次反射后可以回到点 A 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^3$)，表示数据组数。

对于每组数据，一行两个正整数 a, b ($1 \leq a, b < 2^{31}$, $\frac{a}{b} < 90$)，表示 $\angle ABO$ 的角度制大小为 $\frac{a}{b}$ 度。

输出格式

对于每组数据，输出一行一个正整数，表示反射次数。

...

```
#include <bits/stdc++.h>
using namespace std;
int t;
long long a,b,n,m;

int main()
{
    scanf("%d",&t);
    while(t--)
    {
        scanf("%d%d",&a, &b);
        n=180*b-2*a;
        m=360*b;
        printf("%lld\n",m / __gcd(n,m) - 1);
    }
    return 0;
}
```

H 小水獭和科学抗疫 (O 版)

时间限制: 3000ms 内存限制: 65536kb

通过率: 1/2 (50.00%) 正确率: 1/3 (33.33%)

题目描述



小水獭幼儿园出现了疫情!

小水獭幼儿园可以被抽象为一个数轴, 其中共有 n 名学生, 第 i 名学生位于坐标 x_i , 任意两名学生的坐标均不相同。对于每名学生均有 $\frac{1}{2}$ 的概率被感染, 且每名学生是否被感染相互独立。

基于最新的科学研究, 如果第 i 名学生和第 j 名学生都被感染 ($i < j$), 那么需要额外投入 $f(|x_i - x_j|)$ 的核酸费用, 其中 $f(n) = n!$ 。设核酸费用之和的数学期望为 C , 容易证明 $4C$ 是一个整数, 请你求解 $4C$ 对 998244353 取模后的结果。

同学们众志成城, 根据科学精准防控, 一定能战胜疫情!

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 第一行一个正整数 n ($2 \leq n \leq 10^5$), 含义如题目所示。

第二行 n 个正整数 $x_1, x_2, \dots, x_n$ ($1 \leq x_i \leq 10^5$), 含义如题目所示。保证 $x_i \neq x_j$ 对 $1 \leq i < j \leq n$ 成立。

输出格式

对于每组数据, 输出一行一个非负整数表示 $4C$ 对 998244353 取模后的结果。

🐼 \ + + +

```
#include <bits/stdc++.h>
#define maxn 300086

using namespace std;

const double pi = acos(-1);
const int p = 998244353;

struct Complex{
    double x, y;
    Complex(double xx = 0, double yy = 0){
        x = xx, y = yy;
    }
};

Complex operator + (Complex a, Complex b){
    return Complex(a.x + b.x, a.y + b.y);
}

Complex operator - (Complex a, Complex b){
    return Complex(a.x - b.x, a.y - b.y);
}

Complex operator * (Complex a, Complex b){
    return Complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x);
}

int pos[maxn];

void FFT(Complex *a, int lim, int type){
    for(int i = 0; i < lim; i++) if(i < pos[i]) swap(a[i], a[pos[i]]);
    for(int mid = 1; mid < lim; mid <= 1){
        Complex wn(cos(pi / mid), type * sin(pi / mid));
        for(int i = mid < 1, j = 0; j < lim; j += i){
```

```

        Complex w(1, 0);
        for(int k = 0; k < mid; k++, w = w * wn){
            Complex x = a[j + k], y = w * a[j + k + mid];
            a[j + k] = x + y;
            a[j + k + mid] = x - y;
        }
    }
}
if(type == -1) for(int i = 0; i < lim; i++) a[i].x /= lim;
}

int t, n;
Complex a[maxn], b[maxn];
const int m = 1e5;

int main(){
    scanf("%d", &t);
    while(t--){
        for(int i = 0; i < maxn; i++) a[i] = b[i] = {0, 0};
        scanf("%d", &n);
        while(n--){
            int x;
            scanf("%d", &x);
            a[x].x++, b[m - x].x++;
        }
        int lim = 1;
        while(lim <= m * 2) lim <<= 1;
        for(int i = 0; i < lim; i++) pos[i] = (pos[i >> 1] >> 1) | ((i & 1) ? (lim
>> 1) : 0);
        FFT(a, lim, 1), FFT(b, lim, 1);
        for(int i = 0; i < lim; i++) a[i] = a[i] * b[i];
        FFT(a, lim, -1);
        int ans = 0;
        for(int i = m - 1, sum = 1; ~i; i--, sum = 111 * sum * (m - i) % p){
            ans = (ans + 111 * sum * (int)(a[i].x + 0.5)) % p;
        }
        printf("%d\n", ans);
    }
}

```

I 小水獭和 -300IQ

时间限制: 1000ms 内存限制: 65536kb

通过率: 3/4 (75.00%) 正确率: 3/9 (33.33%)

题目描述



小水獭的 IQ 是 -300, 你能帮它提升 IQ 嘛?

给定字符串 s , 设它的长度为 $|s|$, s_i 表示 s 的第 i 个字符。

定义 $S_i = s_1 s_2 \cdots s_{i-1} s_{i+1} \cdots s_{|s|-1} s_{|s|}$ ($1 \leq i \leq |s|$), 相当于把 s 第 i 个字符删掉得到的字符串。

现在, 请你对 $S_1, S_2, \cdots, S_{|s|}$ 按字典序从小到大排序。特别地, 如果两个字符串 $S_i = S_j$ ($i < j$), 令 S_i 的排名靠前。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 一行一个由小写字母组成的字符串 s ($2 \leq |s| \leq 10^5$), 含义同题目描述。

输出格式

对于每组数据, 输出一行 $|s|$ 个正整数, 第 i ($1 \leq i \leq |s|$) 个正整数表示 S_i 的排名。

样例输入

```
#include <stdio.h>
#include <string.h>

#define maxn 100086

char s[maxn], a[maxn];
int rk[maxn];

int main()
{
    int t, n, tmp, i, j, l, r, x;
    scanf("%d", &t);
    while (t--)
    {
        scanf("%s", s + 1), n = strlen(s + 1);
        for (i = n; i; i--)
        {
            if (s[i] == s[i + 1])
                a[i] = a[i + 1];
            else
                a[i] = s[i + 1];
        }
        l = 1, r = n;
        for (i = 1; i <= n; i++)
        {
            if (s[i] > a[i])
                rk[i] = l++;
            else
                rk[i] = r--;
        }
        x = 0;
        for (i = 1; i <= n + 1; i++)
        {
            if (s[i] ^ s[i - 1])
            {
                if (x && rk[x] > rk[i - 1])
```

```

    {
        for (j = 1; j < i - x + 1 - j; j++)
        {
            tmp = rk[x - 1 + j];
            rk[x - 1 + j] = rk[x - 1 + i - x + 1 - j];
            rk[x - 1 + i - x + 1 - j] = tmp;
        }
    }
    x = i;
}
}
for (i = 1; i <= n; i++)
    printf("%d ", rk[i]);
puts("");
}
return 0;
}

```

J

J 这是一道凸包？

时间限制：500ms 内存限制：65536kb

通过率：1/2 (50.00%) 正确率：1/3 (33.33%)

题目描述

Bellaiabella 刚刚学习了计算几何中的凸包知识，她想到了这样一个问题。

二维平面中有 n 个点，第 i 个点的坐标为 (x_i, y_i) ，权值为 w_i 。Bellalabella 可以选择一个点集 S ，设 m 和 M 分别为点集 S 中点的横坐标最小值和最大值，此方案的权值为 S 中所有点与 $(m, 0), (M, 0)$ 构成的凸包面积减去选取点的权值和。特别地，如果 $S = \emptyset$ 则权值为 0。

Bellalabella 想知道所有选取方案中最大权值为多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 5 \times 10^3$)，含义同题目描述。

接下来 n 行，每行三个非负整数 x_i, y_i, w_i ($0 \leq x_i, y_i, w_i \leq 10^4$)，含义同题目描述。

可能存在两个不同的点坐标相同。

输出格式

对于每组数据，输出一行一个浮点数，保留一位小数，表示最大权值。

输入样例

```

1
3
1 2 1
2 4 2
3 2 1

```

输出样例

```

2.0

```

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
int n,k;
const int N = 5010;
vector<tuple<ll,ll,ll>> tmp;
long double dp[N];
long double calc(int l,int r) {

```

```

        return 1.0L*(get<1>(tmp[l-1])+get<1>(tmp[r-1]))*(get<0>(tmp[r-1])-get<0>
(tmp[l-1]))/2-get<2>(tmp[r-1]);
    }
    int main() {

        int t;
        scanf("%d", &t);
        while(t-->0) {
            scanf("%d",&n);
            for(int i = 1; i <= n; i++) dp[i] = 0;
            tmp.clear();
            for(int i=0; i<n; i++) {
                ll x,w,c;
                scanf("%lld %lld %lld",&x,&c,&w);
                tmp.push_back(make_tuple(x,c,w));
            }
            sort(tmp.begin(),tmp.end());
            for(int i=1; i<=n; i++) {
                dp[i]=-get<2>(tmp[i-1]);
            }
            for(int i=1; i<=n; i++) {
                for(int j=1; j<i; j++) {
                    dp[i]=max(dp[i],dp[j]+ calc(j,i));
                }
            }
            long double tmpa=0;
            for(int i=1; i<=n; i++) {
                tmpa=max(tmpa,dp[i]);
            }
            printf("%.1Lf\n",tmpa);
        }
    }
}

```

C7

A

第一题

对于排序算法和时间复杂度分析，下列说法正确的是__。

- A. 根据排序元素所在位置的不同，排序分内排序和外排序。
- B. 可以使用 Amortized Analysis（平摊分析）对归并排序进行复杂度分析。
- C. 可以使用概率分析方法对快速排序进行复杂度分析。
- D. 堆排序是一种稳定的排序算法。
- E. 当上下限表达式相等时，可以使用 O 表示法来描述算法代价。

第二题

对于动态规划，下列说法正确的是__。

- A. 动态规划可以解决最优二叉搜索树的问题，其中最优二叉搜索树指的是最坏查找时间最小的二叉搜索树。
- B. 可以使用动态规划解决的问题只需要满足最优化原理即可，不需要考虑是否有后效性。
- C. Strassen 矩阵乘法问题可以使用动态规划来解决。
- D. 动态规划通常以自底向上的方式求解最优解。

第三题

对于 **NP** 问题，下列说法正确的是__。

- A. **NP** 问题都是不可能解决的问题。
- B. **P** 问题包含在 **NP** 问题中。
- C. **NP** 完全问题是 **P** 问题的子集。
- D. **NP** 问题包含在 **P** 问题中。

第四题

对 n ($5 \times 10^3 < n < 10^4$) 个点的点集，采用 Graham's scan 算法求凸包，下列说法正确的是__。

- A. Graham's scan 算法的计算复杂度为 $O(kn)$ ，其中 k 为凸包顶点个数。
- B. 可以用 The accounting method 方法有效分析 Graham's scan 的计算复杂度。
- C. 不可以用 Aggregate analysis 方法有效分析 Graham's scan 的计算复杂度。
- D. 凸包顶点个数 k 约为 $\frac{n}{5}$ 时，Graham's scan 算法比 Jarvis's march 更快。

B

B XIAO7 和搬砖

时间限制：1000ms 内存限制：65536kb

通过率：37/106 (34.91%) 正确率：37/525 (7.05%)

题目描述

XIAO7 准备放弃猫娘乐园专心搬砖。

有 N 种砖头，第 i 种砖头有 m_i 块，每一块的价值为 d_i ，每一个单位时间（从 1 开始编号）XIAO7 都会搬一块砖，直到无砖可搬为止。

同时，有一个得分系数 F ，初始时为 1。搬一块砖的得分为当时的得分系数乘以对应砖的价值。

此外，有 T 个时间分割点，每经过一个时间分割点，即从时间分割点的下一个单位时间开始， F 就会增加 1。

请你告诉 XIAO7 她能够获得的最大得分是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 N ($1 \leq N \leq 10^5$)，含义同题目描述。

接下来 N 行，每行两个正整数 m_i, d_i ($1 \leq m_i \leq 10^5, 1 \leq d_i \leq 10^3$)，含义同题目描述。

接下来一行一个正整数 T ($1 \leq T \leq 10^5$)，含义同题目描述。

接下来一行 T 个互不相同的正整数 p_i ($1 \leq p_i \leq 10^9$)，表示时间分割点，即从第 $p_i + 1$ 个单位时间开始 F 会增加 1。

输出格式

对于每组数据，输出一行一个正整数，表示最大得分。

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<map>
#include<assert.h>
using namespace std;
typedef long long ll;
```

```

typedef pair<ll, ll> pi;
const int N = 100005;
const ll inf = 1ll << 60;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
pi a[N];
ll t[N];
int main() {

    int n;
    int tt;
    scanf("%d", &tt);
    while(tt--) {
        scanf("%d", &n);
        assert(n <= 1e5);
        for(int i = 1; i <= n; i++) {
            a[i].second = getint();
            a[i].first = getint();
        }
        sort(a + 1, a + n + 1);

        int T = getint();
        for(int i = 1; i <= T; i++) {
            t[i] = getint();
        }
        sort(t + 1, t + T + 1), t[T + 1] = inf;

        ll ans = 0, sum = 0;
        for(int i = 1, j = 1; i <= n; ) {
            if(sum + a[i].second <= t[j]) {
                ans += 1ll * j * a[i].first * a[i].second;
                sum += a[i].second, i++;
            } else {
                ll m = t[j] - sum;
                a[i].second -= m;
                ans += 1ll * j * a[i].first * m;
                sum += m, j++;
            }
        }

        cout << ans << endl;
    }

    return 0;
}

```

C Bellalabella 与中位数

时间限制: 2000ms 内存限制: 65536kb

通过率: 135/172 (78.49%) 正确率: 135/411 (32.85%)

题目描述

Bellalabella 学到了中位数的知识, Bellalabella 想知道对于一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 所有前缀的中位数是多少。

由于 Bellalabella 学艺不精, 不知道长度为偶数的序列的中位数的定义, 因此她只需要你求出长度为奇数的所有前缀的中位数。

一个长度为 $2n - 1$ 的序列的中位数是将这个序列从小到大排序后第 n 个位置的数, 例如 $[1, 3, 2]$ 中位数为 2 , $[2, 3, 2]$ 中位数为 2 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$), 表示数据组数。

对于每组数据, 第一行一个正奇数 n ($1 \leq n \leq 10^5$), 含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^5$), 含义同题目描述。

对于得分占比 50% 的测试点, 保证 $0 \leq a_i \leq 20$ 。

输出格式

对于每组数据, 输出一行 $\frac{n+1}{2}$ 个整数, 第 i 个数表示长度为 $2i - 1$ 的前缀的中位数。

输入样例

```
2
5
3 4 5 1 2
13
13 25 19 11 29 19 17 17 5 9 11 11 3
```

输出样例

```
3 4 3
13 19 19 19 17 17 13
```

```
#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 1000005
#define ls (tot << 1)
#define rs (tot << 1 | 1)
#define PII pair<int, int>
#define fi first
#define se second
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
const double pi = acos(-1.0L);
const int mod = 998244353;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int n, a[maxn];
priority_queue<int> q;
priority_queue<int, vector<int>, greater<int>> p;

void solve(){
    while(!q.empty())q.pop();
```

```

while(!p.empty())p.pop();
n = read();
for(int i = 1; i <= n; i++) {
    a[i] = read();
    q.push(a[i]);
    if(i & 1){
        while(q.size() > p.size()) p.push(q.top()), q.pop();
        while(q.size() < p.size()) q.push(p.top()), p.pop();
        printf("%d ", q.top());
    }
}
puts("");
}

int main(){

    int t = read();
    while(t--){
        solve();
    }

    return 0;
}

```

D

D Bellalabella 与「三分之一」

时间限制: 1000ms 内存限制: 65536kb

通过率: 1/16 (6.25%) 正确率: 1/55 (1.82%)

题目描述

Bellalabella 在自己进行游戏，游戏规则如下：

- 游戏总共进行 n 轮，每一轮都由 Bellalabella 先手。
- 初始 Bellalabella 有一个数字 0。
- 每一轮 Bellalabella 可以从 A, B, C 三个数中选择一个数，并把这个数加到自己的数字上。
- 设最终 Bellalabella 手中的数字设为 x 。
- 如果对于其他所有选择方式的出的结果 y 都有 $\left| x - \frac{n(A+B+C)}{3} \right| \leq \left| y - \frac{n(A+B+C)}{3} \right|$ ，那么 Bellalabella 就会获胜。

简单来说，即 Bellalabella 想要让自己最后的数字最接近 $\frac{n(A+B+C)}{3}$ ，Bellalabella 想知道自己最后的数字是多少才能赢，容易证明这个解是唯一的。

注意在 n 轮游戏中， A, B, C 三个数的值是固定的。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行四个正整数 n, A, B, C ($1 \leq n \leq 10^5$, $1 \leq A, B, C \leq 10^9$)，含义同题目描述。

对于得分占比 50% 的测试点保证 $1 \leq n \leq 100$ 且 $1 \leq A, B, C \leq 100$ 。

输出格式

对于每组数据，输出一行一个正整数，表示 Bellalabella 能够获得的最接近 $\frac{n(A+B+C)}{3}$ 的数为多少。

```

#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 100005
#define ls (tot << 1)
#define rs (tot << 1 | 1)

```

```

#define PII pair<int, int>
#define fi first
#define se second
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
const double pi = acos(-1.0L);
const int mod = 998244353;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

ll n, A, B, C;

void solve(){
    ll ans = 0;

    n = read(), A = read(), B = read(), C = read();
    if(A > B) swap(A, B);if(A > C) swap(A, C);if(B > C) swap(B, C);
    ll sum = n * (A + B + C);
    for(int i = 0; i <= n; i++){
        ll mn = 1ll * A * i + 1ll * B * (n - i);
        if(mn * 3 >= sum || B == C){
            if(abs(ans * 3 - sum) > abs(mn * 3 - sum)) ans = mn;
        }else{
            ll delta = (C - B) * 3, nd = sum - mn * 3;
            ll down = nd / delta, up = down + 1;
            if(down >= n - i) down = up = n - i;
            mn = 1ll * A * i + 1ll * B * (n - i - down) + 1ll * C * down;
            if(abs(ans * 3 - sum) > abs(mn * 3 - sum)) ans = mn;
            mn = 1ll * A * i + 1ll * B * (n - i - up) + 1ll * C * up;
            if(abs(ans * 3 - sum) > abs(mn * 3 - sum)) ans = mn;
        }
    }
    printf("%lld\n", ans);
}

int main(){
    int t = read();
    while(t--){
        solve();
    }

    return 0;
}

```

E

本题采用「炉石传说」背景，无论您是否玩过这个游戏，请完全以本题规则为准。

小水獭和奥布莱克斯每人各有 7 个随从，编号为 1 到 7，每个随从有两个基本属性：攻击力和生命值；以及可能拥有的三种特殊能力：**嘲讽**、**圣盾**、**亡语**，这三种能力会在下方解释清楚。

每个随从从受到伤害时生命值会减少相应的数值，如果一个随从的生命值小于等于 0，则该随从死亡。

小水獭和奥布莱克斯会轮流执行下列操作，直到其中一方（或两方）的全部随从死亡，小水獭先手：

- 选择己方攻击力最高的一个未死亡随从，如果有多个随从攻击力相同，选择编号小的那一个；记这个随从的编号为 x 。
- 如果对方有具有嘲讽能力的未死亡随从，则选择对方生命值最高的一个具有嘲讽能力的未死亡随从，否则选择对方生命值最高的一个未死亡随从，如果有多个随从生命值相同，选择编号小的那一个；记这个随从的编号为 y 。
- 对 x 号随从造成等同于 y 号随从从攻击力的伤害，对 y 号随从造成等同于 x 号随从从攻击力的伤害。

对于上述三种特殊能力的解释：

- 具有嘲讽能力的随从，会被对方优先攻击，已在上述游戏规则阐述过。
- 具有圣盾能力的随从，第一次受到伤害时生命值不会减少，并失去圣盾能力。
- 具有亡语能力的随从，会在死亡后额外产生一个和它编号相同的随从，该随从的攻击力和生命值会在输入中给出，且不具有任何特殊能力。

莫卡正在和教练同学学习术学，于是她想问问你，在上述游戏过程中，存不存在一个时刻（包括开始时刻和结束时刻），使得场上未死亡随从数量大于 2，且通过名为**教科书般的毁灭**的下列操作可以使得场上不存在任何未死亡随从。

- 对所有随从造成 1 点伤害，如果这次操作使得至少一个随从死亡，重复这个操作。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，共 14 行，分别描述小水獭的 1 到 7 号随从以及奥布莱克斯的 1 到 7 号随从。

每行 6 个整数 a, b, c, d, e, f ($1 \leq a, b \leq 20, 0 \leq c, d \leq 1, 0 \leq e, f \leq 5$)，具体说明如下：

- a 表示该随从的攻击力。
- b 表示该随从的生命值。
- 如果 $c = 1$ 说明该随从具有嘲讽能力，否则该随从没有嘲讽能力。
- 如果 $d = 1$ 说明该随从具有圣盾能力，否则该随从没有圣盾能力。
- 如果 $e = f = 0$ 说明该随从不具有亡语能力，否则该随从具有亡语能力，且死亡后产生随从的攻击力和生命值分别为 e 和 f ，保证两者同时为 0 或同时非 0。

输出格式

对于每组数据，输出一行一个整数，如果存在一个时刻，使得场上未死亡随从数量大于 2，且通过教科书般的毁灭可以使得场上不存在任何未死亡随从，输出 1；否则，输出 0。

```
#include <stdio.h>
#define max(a, b) (a > b ? a : b)

struct Minion
{
    int a, b, c, d, e, f;
} A[2][10];

int check()
{
    int c[30] = {0}, mx = 0, i, j, cnt = 0;
    for (i = 0; i <= 1; i++)
    {
        for (j = 1; j <= 7; j++)
        {
            if (A[i][j].b <= 0) continue;
            cnt++;
            c[A[i][j].b + A[i][j].d] = 1;
            c[A[i][j].b + A[i][j].d + A[i][j].f] = 1;
            mx = max(mx, A[i][j].b + A[i][j].d + A[i][j].f);
        }
    }
    if (cnt <= 2) return 0;
    for (i = 1; i <= mx; i++)
        if (!c[i]) return 0;
    return 1;
}

int main()
{
    int t, i, j, suc, x, y;
```

```

scanf("%d", &t);
while (t--)
{
    for (i = 0; i <= 1; i++)
        for (j = 1; j <= 7; j++) scanf("%d%d%d%d%d", &A[i][j].a, &A[i][j].b, &A[i][j].c, &A[i][j].d, &A[i][j].e, &A[i][j].f);
    suc = check();
    for (i = 0; !suc; i ^= 1)
    {
        x = y = 0;
        for (j = 1; j <= 7; j++)
        {
            if (A[i][j].b > 0 && (!x || A[i][j].a > A[i][x].a))
            {
                x = j;
            }
        }
        for (j = 1; j <= 7; j++)
        {
            if (A[i ^ 1][j].b > 0 && (!y || A[i ^ 1][j].c > A[i ^ 1][y].c || A[i ^ 1][j].c == A[i ^ 1][y].c && A[i ^ 1][j].b > A[i ^ 1][y].b))
            {
                y = j;
            }
        }
        if (!x || !y) break;
        if (A[i][x].d)
        {
            A[i][x].d = 0;
        }
        else
        {
            A[i][x].b -= A[i ^ 1][y].a;
        }
        if (A[i ^ 1][y].d)
        {
            A[i ^ 1][y].d = 0;
        }
        else
        {
            A[i ^ 1][y].b -= A[i][x].a;
        }
        if (A[i][x].b <= 0 && A[i][x].e)
        {
            A[i][x].a = A[i][x].e, A[i][x].b = A[i][x].f;
            A[i][x].c = A[i][x].d = A[i][x].e = A[i][x].f = 0;
        }
        if (A[i ^ 1][y].b <= 0 && A[i ^ 1][y].e)
        {
            A[i ^ 1][y].a = A[i ^ 1][y].e, A[i ^ 1][y].b = A[i ^ 1][y].f;
            A[i ^ 1][y].c = A[i ^ 1][y].d = A[i ^ 1][y].e = A[i ^ 1][y].f = 0;
        }
        suc |= check();
    }
    puts(suc ? "1" : "0");
}

```

```
    return 0;
}
```

F

题目描述



小水獭正在前往加帕里幼儿园学习构造题的相关技巧。

对于任意有限数集 A, B ，定义二者之和 $A + B$ 为：

$$A + B = \{x + y \mid x \in A, y \in B\}$$

例如，当 $A = \{1, 2\}, B = \{3, 4\}$ ，有 $A + B = \{4, 5, 6\}$ ，注意此处并非可重集，因此即使 $1 + 4 = 2 + 3 = 5$ ， $A + B$ 也仅包含一个 5。

记有限数集 A 中的元素个数为 $|A|$ 。

现给定整数集合 S ，记 $T = S + S$ ，请你求解 $|S + T|$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 $|S|$ ($1 \leq |S| \leq 8 \times 10^4 + 1$)，含义如题目所示。

第二行 $|S|$ 个互不相同的整数 $a_1, a_2, \dots, a_{|S|}$ ($0 \leq |a_i| \leq 4 \times 10^4$)，表示 S 中的元素。

输出格式

对于每组数据，输出一行一个正整数表示 $|S + T|$ 。

```
#include <bits/stdc++.h>
#define maxn 300086

using namespace std;

#define double long double

const double pi = acos(-1.0L);
const int p = 998244353;

struct Complex{
    double x, y;
    Complex(double xx = 0, double yy = 0){
        x = xx, y = yy;
    }
};

Complex operator + (Complex a, Complex b){
    return Complex(a.x + b.x, a.y + b.y);
}

Complex operator - (Complex a, Complex b){
    return Complex(a.x - b.x, a.y - b.y);
}

Complex operator * (Complex a, Complex b){
    return Complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x);
}
```

```

}

int pos[maxn];

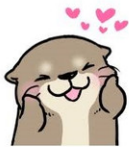
void FFT(Complex *a, int lim, int type){
    for(int i = 0; i < lim; i++) if(i < pos[i]) swap(a[i], a[pos[i]]);
    for(int mid = 1; mid < lim; mid <= 1){
        Complex wn(cos(pi / mid), type * sin(pi / mid));
        for(int i = mid < 1, j = 0; j < lim; j += i){
            Complex w(1, 0);
            for(int k = 0; k < mid; k++, w = w * wn){
                Complex x = a[j + k], y = w * a[j + k + mid];
                a[j + k] = x + y;
                a[j + k + mid] = x - y;
            }
        }
    }
    if(type == -1) for(int i = 0; i < lim; i++) a[i].x /= lim;
}

int t, n;
Complex a[maxn];
const int m = 4e4;

int main(){
    scanf("%d", &t);
    while(t--){
        for(int i = 0; i < maxn; i++) a[i] = {0, 0};
        scanf("%d", &n);
        while(n--){
            int x;
            scanf("%d", &x);
            a[x + m].x++;
        }
        int lim = 1;
        while(lim <= m * 6) lim <= 1;
        for(int i = 0; i < lim; i++) pos[i] = (pos[i >> 1] >> 1) | ((i & 1) ? (lim
>> 1) : 0);
        FFT(a, lim, 1);
        for(int i = 0; i < lim; i++) a[i] = a[i] * a[i] * a[i];
        FFT(a, lim, -1);
        int ans = 0;
        for(int i = 0; i < lim; i++) ans += ((long long)(a[i].x + 0.5)) > 0;
        printf("%d\n", ans);
    }
}

```

题目描述



小水獭正在两个 $n \times m$ 的矩阵中随机游走，记这两个矩阵分别为 A 和 B 。

具体来说，小水獭会首先选中两个矩阵中的两个位置 (x_1, y_1) 和 (x_2, y_2) ，之后进行任意次如下四种操作之一：

- 将 (x_1, y_1) 变为 $(x_1 - 1, y_1)$ ，将 (x_2, y_2) 变为 $(x_2 - 1, y_2)$ 。
- 将 (x_1, y_1) 变为 $(x_1 + 1, y_1)$ ，将 (x_2, y_2) 变为 $(x_2 + 1, y_2)$ 。
- 将 (x_1, y_1) 变为 $(x_1, y_1 - 1)$ ，将 (x_2, y_2) 变为 $(x_2, y_2 - 1)$ 。
- 将 (x_1, y_1) 变为 $(x_1, y_1 + 1)$ ，将 (x_2, y_2) 变为 $(x_2, y_2 + 1)$ 。

任意时刻，小水獭需要保证 $1 \leq x_1, x_2 \leq n$ 且 $1 \leq y_1, y_2 \leq m$ ，同时 $A_{x_1, y_1} = B_{x_2, y_2}$ 。

记 S 为小水獭在 A 所经过位置对应元素组成的集合，那么最理想情况下 S 中元素数量 $|S|$ 的最大值是多少呢？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq nm \leq 10^5$)，含义如题目所示。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个正整数 $A_{i,j}$ ($1 \leq A_{i,j} \leq nm$)，表示第一个矩阵 A 中的元素。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个正整数 $B_{i,j}$ ($1 \leq B_{i,j} \leq nm$)，表示第二个矩阵 B 中的元素。

保证 $A_{i,j} = A_{i',j'} \Leftrightarrow (i = i' \wedge j = j')$ 且 $B_{i,j} = B_{i',j'} \Leftrightarrow (i = i' \wedge j = j')$ 。

输出格式

对于每组数据，输出一行一个正整数最理想情况下 $|S|$ 的最大值。

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
int a[maxn];
pair<int, int> b[maxn], c[maxn];
bool vis[maxn];

int id(int x, int y){
    return (x - 1) * m + y;
}

const int dx[4] = {0, 0, 1, -1}, dy[4] = {1, -1, 0, 0};

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++) scanf("%d", &a[id(i, j)]);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++){
            int x;
            scanf("%d", &x);
            b[x] = {i, j};
        }
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++){
            c[id(i, j)] = {i - b[a[id(i, j)]]>.first, j - b[a[id(i, j)]]>.second};
        }
        for(int i = 1; i <= n * m; i++) vis[i] = false;
        int ans = 0;
        for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++) if(!vis[id(i, j)]){
            int sum = 0;
```

```

queue<pair<int, int> > q;
vis[id(i, j)] = true, q.push({i, j});
while(!q.empty()){
    int x = q.front().first, y = q.front().second;q.pop();
    sum++;
    for(int k = 0;k < 4;k++){
        int xx = x + dx[k], yy = y + dy[k];
        if(xx >= 1 && xx <= n && yy >= 1 && yy <= m){
            if(c[id(x, y)] == c[id(xx, yy)] && !vis[id(xx, yy)])
vis[id(xx, yy)] = true, q.push({xx, yy});
        }
    }
}
ans = max(ans, sum);
}
printf("%d\n", ans);
}
}

```

H

小水獭定义了一种奇怪的最短路。

形式化来说，给定一个 n 个点 m 条边的带权无向连通图，对于一条至少经过一条边的路径，设其所有边的权值之和为 A ，边权中的最大值和最小值为 B 和 C ，则定义这条路径的权值为 $A - B + C$ 。

小水獭想知道，从节点 1 到其它每个节点在本题路径权值定义下的最短路长度，你能帮帮它嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($2 \leq n \leq 10^5$, $n - 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n$, $0 \leq w_i \leq 10^4$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

保证图是连通的，且没有自环和重边。

输出格式

对于每组数据，输出一行 $n - 1$ 个非负整数，表示从节点 1 到节点 $2, 3, \dots, n$ 在本题路径权值定义下的最短路长度。

输入样例

```

2
7 6
1 2 1
2 3 2
1 4 2
4 5 1
1 6 3
6 7 3
4 6
1 2 1
1 3 2
1 4 3
2 3 4
2 4 5
3 4 6

```

输出样例

```

1 2 2 2 3 6
1 2 2

```

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
vector<pair<int, int> > v[maxn];
int d[maxn][4];
bool vis[maxn][4];

```

```

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) v[i].clear();
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            v[x].push_back({y, z}), v[y].push_back({x, z});
        }
        priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>,
greater<tuple<int, int, int>>> q;
        for(int i = 1; i <= n; i++) for(int j = 0; j < 4; j++) d[i][j] = 0x3f3f3f3f,
vis[i][j] = false;
        d[1][0] = 0, q.push(make_tuple(0, 1, 0));
        while(!q.empty()){
            int x = get<1>(q.top()), y = get<2>(q.top()); q.pop();
            if(vis[x][y]) continue;
            vis[x][y] = true;
            for(auto it : v[x]){
                int to = it.first, co = it.second;
                if(d[to][y] > d[x][y] + co){
                    d[to][y] = d[x][y] + co;
                    q.push(make_tuple(d[to][y], to, y));
                }
                if(!(y & 1) && d[to][y | 1] > d[x][y]){
                    d[to][y | 1] = d[x][y];
                    q.push(make_tuple(d[to][y | 1], to, y | 1));
                }
                if(!(y & 2) && d[to][y | 2] > d[x][y] + co * 2){
                    d[to][y | 2] = d[x][y] + co * 2;
                    q.push(make_tuple(d[to][y | 2], to, y | 2));
                }
                if(!(y & 3) && d[to][y | 3] > d[x][y] + co){
                    d[to][y | 3] = d[x][y] + co;
                    q.push(make_tuple(d[to][y | 3], to, y | 3));
                }
            }
        }
        for(int i = 2; i <= n; i++) printf("%d ", d[i][3]), assert(d[i][3] <
0x3f3f3f3f); puts("");
    }
}

```

题目描述



小水獭正在预习计算几何。

二维平面上有 n 个不同的点，小水獭请你帮它计算这 n 个点所组成的集合有多少个非空子集满足其中点构成的凸包（有可能退化为一条线段或一个点）严格不包含原点，即原点不能在凸包内或凸包上。

由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($3 \leq n \leq 10^5$)，含义同题目描述。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$, $|x_i| + |y_i| \neq 0$)，表示第 i 个点的坐标。

保证任意两点坐标不同。

对于得分占比 50% 的测试点，保证 $1 \leq n \leq 15$ 。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 998 244 353 取模后的结果。

```
#include <bits/stdc++.h>
#define maxn 200086

using namespace std;

typedef long long ll;

struct node{
    ll x, y;
    int f, id;
    void getf(){
        if(x >= 0 && y > 0) f = 1;
        else if(x < 0 && y >= 0) f = 2;
        else if(x <= 0 && y < 0) f = 3;
        else f = 4;
    }
    bool operator < (const node & a) const {return f < a.f || (f == a.f && x * a.y > y * a.x) || (f == a.f && x * a.y == y * a.x && id < a.id);}
}a[maxn];

const int p = 998244353;

int t, n;
int x[maxn], y[maxn];
int pw[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        pw[0] = 1;for(int i = 1;i <= n;i++) pw[i] = pw[i - 1] * 2 % p;
        for(int i = 1;i <= n;i++) scanf("%lld%lld", &a[i].x, &a[i].y),
a[i].getf(), a[i].id = i;
        sort(a + 1, a + n + 1);
        for(int i = n + 1;i <= n * 2;i++) a[i] = a[i - n];
        int x = 1, ans = 0;
        for(int i = 1;i <= n;i++){
            while(x + 1 < i + n && (a[x + 1].x * a[i].y - a[i].x * a[x + 1].y == 0 && a[x + 1].f == a[i].f
                || a[x + 1].x * a[i].y - a[i].x * a[x + 1].y < 0)){
```

```

        x++;
    }
    ans = (ans + pw[x - i]) % p;
}
printf("%d\n", ans);
}
}

```

J

小水獭正在预习字符串。

给定 n 个字符串，小水獭想知道所有满足这 n 个字符串均是其子串的字符串中，最短字符串的长度是多少？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 18$)，含义同题目描述。

接下来 n 行，每行一个由小写字母组成的字符串 s_i ($1 \leq |s_i| \leq 10^4$)，表示第 i 个字符串。

对于得分占比 50% 的测试点，保证 $1 \leq n \leq 6$ 。

输出格式

对于每组数据，输出一行一个正整数，表示满足条件的最短字符串长度。

输入样例

```

2
4
aba
bab
bcc
cbaab
5
aaaaa
aaaa
aaa
aa
a

```

输出样例

```

9
5

```

```

#include <bits/stdc++.h>
#define maxn 10086

using namespace std;

char s[18][maxn];
int len[18];
int nxt[maxn];

int cal(int x, int y){
    for(int i = 1; i < len[x]; i++){
        int j = nxt[i - 1];
        while(j > 0 && s[x][i] != s[x][j]) j = nxt[j - 1];
        if(s[x][i] == s[x][j]) j++;
        nxt[i] = j;
    }
    int j = 0;
    for(int i = 0; i < len[y]; i++){
        while(j > 0 && s[x][j] != s[y][i]) j = nxt[j - 1];
        if(s[x][j] == s[y][i]) j++;
        if(j == len[x]) return -1;
    }
    return j;
}

```

```

int t, n;
int a[18][18];
int f[1 << 18][18];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 0; i < n; i++) scanf("%s", s[i]), len[i] = strlen(s[i]);
        vector<int> v;
        for(int i = 0; i < n; i++){
            bool tag = false;
            for(int j = 0; j < n; j++) if(i ^ j){
                a[j][i] = cal(i, j);
                if(a[j][i] == -1){
                    if(len[i] < len[j] || i < j){
                        tag = true;
                        break;
                    }
                }
            }
            if(tag) continue;
            v.push_back(i);
        }
        int m = v.size();
        for(int i = 0; i < (1 << m); i++) for(int j = 0; j < m; j++) f[i][j] = 1e9;
        for(int i = 0; i < m; i++) f[1 << i][i] = len[v[i]];
        for(int i = 0; i < (1 << m); i++) for(int j = 0; j < m; j++){
            for(int k = 0; k < m; k++) if(!(i & (1 << k))){
                f[i | (1 << k)][k] = min(f[i | (1 << k)][k], f[i][j] + len[v[k]]
- a[v[j]][v[k]]);
            }
        }
        int ans = 1e9;
        for(int i = 0; i < m; i++) ans = min(ans, f[(1 << m) - 1][i]);
        printf("%d\n", ans);
    }
}

```